Day 4

Accessible Front-End Delivery

Accessibility goes beyond individual controls. Data, visuals, motion, mobile, and content all shape the experience — and final delivery must be both testable and usable by everyone.

DAYS 4 & 5 | FRONT-END ACCESSIBILITY

COURSE PROGRESS

Where We Are Now

This course has been building a complete picture of front-end accessibility. Each day has added a new layer of responsibility to your implementation practice.

01

Day 1 — Semantic Structure

Landmarks, headings, roles, and meaningful HTML that communicates document shape to assistive technology.

02

Day 2 — Forms and Validation

Accessible labels, error handling, live regions, and the relationship between inputs and their context.

03

Day 3 — Interaction and Dialogs

Keyboard patterns, focus management, route change announcements, and modal accessibility.

TODAY'S AGENDA

Today's Focus

Days 4 and 5 address the parts of an interface that are most often overlooked: structured data, visual systems, motion, mobile context, content quality, and readiness for final delivery.

Accessible Data Displays

Tables, charts, sorting, pagination, and responsive data patterns.

Visual Systems and Motion

Design tokens, focus indicators, colour and state, animation, and async feedback.

Mobile, Content, and Final Review

Touch targets, plain language, semantic SEO, frameworks, and delivery readiness.

ACCESSIBLE DATA

Accessible Data Displays

The Core Principle

Data must remain understandable regardless of how it is rendered. Structure carries meaning that visual layout alone cannot — especially for users of screen readers and other assistive technology.

Three Guiding Rules

- **Structure over appearance** — semantic markup conveys relationships
- **Context before details** — users need orientation first
- **Multiple access paths** — visual, textual, and structured

Accessible data is not about adding descriptions after the fact — it is about building data structures that communicate meaning natively.

ACCESSIBLE DATA

Tables

Tables are the correct semantic tool for tabular data. Avoid recreating table layouts using `div` elements with CSS — the visual output may appear identical, but the structural meaning is lost entirely for assistive technology users.

Use Real Table Markup

Use `<table>`, `<thead>`, `<tbody>`, `<tr>`, `<th>`, and `<td>`. These elements carry built-in semantics that screen readers depend on.

Add Captions and Headers

A `<caption>` element provides the table's purpose before data is read. Column headers using `<th>` establish the axis of each data point.

Never Fake a Table

A grid of `div` elements styled to look like a table provides no semantic row/column relationships. Screen readers will read cells without any awareness of their position.

ACCESSIBLE DATA

Table Accessibility Essentials

Three elements form the foundation of an accessible table: a meaningful caption, correctly scoped headers, and clear row and column relationships throughout the entire dataset.

Element	Purpose	Implementation Note
`<caption>`	Describes the table's purpose	Placed as first child of `<table>`
`<th scope="col">`	Identifies a column header	Use in `<thead>` rows
`<th scope="row">`	Identifies a row header	Use in first cell of data rows
`headers` attribute	Links cell to multiple headers	Required for complex or merged cells

In React or Angular, always verify that component libraries render actual `<th>` elements with correct `scope` attributes — not just visually styled divs.

ACCESSIBLE DATA

Sorting and Pagination

Sorting Controls

- Sort triggers must be real `<button>` elements, not clickable text or icons
- Current sort direction must be communicated via `aria-sort="ascending"` or `"descending"` on the column header
- After sorting, focus should remain on the activated control or move predictably

Pagination Controls

- Use a `<nav>` landmark with a descriptive `aria-label`
- Indicate the current page with `aria-current="page"`
- After navigating pages, move focus to the new content region or announce the change via a live region

⚠ Never rely solely on visual cues like arrow icons to communicate sort direction — the state must exist in the accessibility tree.

ACCESSIBLE DATA

Responsive Data

Adapting data displays for small screens requires preserving the relationship between a value and its label. Breaking this relationship — even visually — creates comprehension barriers for all users and navigational failures for screen reader users.

Horizontal Scroll

Acceptable for complex tables when the scrollable container is keyboard-focusable and its scrollability is communicated. Add `tabindex="0"` and a clear label.

Card Pattern

When rows become cards on mobile, every value must retain its visible label. Never show a value without its field name — e.g. display "Status: Active", not just "Active".

Avoid Destruction

Do not hide columns silently, strip headers, or reorder content purely for visual fit. Each change to structure must be evaluated against its impact on meaning.

ACCESSIBLE DATA

Charts and Visual Data

A chart is a visual representation — not a data source in itself. For users who cannot perceive the visual, the chart communicates nothing unless it is supplemented with accessible alternatives.

Never Make a Chart the Only Source of Insight

If the chart disappears, the insight must still be available. Screen reader users, users with low vision, and users in high-contrast mode all need textual equivalents.

Provide a Text Summary

A short paragraph below the chart describing the key trend or finding is often more useful than any technical alternative approach. Write it as you would explain the chart to a colleague.

Provide Underlying Data When Needed

For complex charts, offer the dataset in a table. This serves screen reader users, power users, and anyone who wants to verify or explore the data independently.

ACCESSIBLE DATA

Chart Alternatives

Implementation Patterns

Headings and Summaries

Use a heading above the chart that names the insight, not just the chart type. "Revenue grew 40% in Q3" is more useful than "Revenue Chart".

Trends Explained in Text

Describe direction, magnitude, and notable exceptions. Place this text directly adjacent to the chart so it is encountered in natural reading order.

Avoid Colour-Only Encoding

Every data series must be distinguishable by more than colour. Use patterns, direct labels, shapes, or text annotations alongside colour.

Quick Reference

- Use `aria-label` or `aria-labelledby` on chart containers
- Associate a visible data table via `aria-details`
- SVG charts: use `<title>` and `<desc>` elements
- Canvas charts require a fully accessible alternative

VISUAL SYSTEMS

Design Tokens and Accessibility

Design tokens are the named values — colours, spacing, font sizes, border radii — that define a design system. Because tokens propagate across every component, their accessibility quality has a multiplying effect: good tokens protect the whole system, bad tokens create failures everywhere.

Colour Tokens

Contrast ratios must meet WCAG 2.1 AA. Text on backgrounds, UI controls on their backgrounds, and focus indicators all require verified ratios.

Spacing Tokens

Adequate spacing prevents touch target failures and visual crowding. Spacing tokens should account for both desktop and mobile interaction contexts.

Focus and State Tokens

Focus ring colour, thickness, and offset should be design token values — not overridden ad hoc. States like disabled, hover, error, and selected need distinct, token-driven treatments.

VISUAL SYSTEMS

Focus Indicators

The Fundamental Rule

Focus must always be visible. Removing `outline: none` without providing a replacement violates WCAG 2.4.7 (Level AA) and leaves keyboard users unable to determine where they are on the page.

Focus design is a first-class design concern — not a developer afterthought.

Implementation Guidance

- Use `:focus-visible` to show focus only for keyboard navigation, not mouse clicks
- Ensure the focus ring has a 3:1 contrast ratio against adjacent colours (WCAG 2.4.11, Level AA in WCAG 2.2)
- Define focus tokens in your design system so they cannot be accidentally overridden per component
- Test focus visibility in both light and dark themes

⊗ Resetting default browser focus styles without a design system replacement is one of the most common and impactful accessibility failures in production interfaces.

VISUAL SYSTEMS

Colour and State

Colour is a powerful visual tool, but it cannot be the *only* tool used to communicate meaning. Approximately 8% of men and 0.5% of women have some form of colour vision deficiency. Colour-only state communication excludes these users entirely.

Error States

Combine red colour with an error icon, a text label ("Error"), and a clear message. Never rely on colour alone to indicate that something has gone wrong.

Selected and Active States

Use shape change, underlines, bold weight, or background fill in addition to colour. Tab components, toggle buttons, and nav items are common failure points.

Disabled States

Disabled controls must still meet minimum contrast for perceivability. Communicate disabled state with text or icon in addition to a reduced colour treatment.

VISUAL SYSTEMS

Motion and Feedback

When Motion Helps

Animation that supports understanding — such as a panel sliding open to reveal its origin, or a progress indicator advancing to show completion — adds genuine value. The motion explains the relationship.

When Motion Harms

Decorative animations, auto-playing carousels, parallax scrolling, and rapid transitions can trigger vestibular disorders and distract users with attention or cognitive differences.

prefers-reduced-motion

Implement the `prefers-reduced-motion` media query in all component animation logic.

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation: none;
    transition: none;
  }
}
```

In React and Angular, check this preference in JavaScript using `window.matchMedia` for programmatic animations.

VISUAL SYSTEMS

Loading and Status States

Asynchronous interfaces introduce a moment of ambiguity — the UI has changed, but the new content is not yet available. This ambiguity must be communicated clearly to all users, including those who cannot see the visual loading indicator.

Communicate Loading Clearly

A skeleton screen or spinner without a text announcement is silent to screen readers. Use an `aria-live="polite"` region to announce when loading begins and when content has loaded.

Provide Meaningful Context

Announcements like "Loading results…" and "10 results loaded" are far more useful than generic spinners. Name what is loading and confirm when it is complete.

Avoid Focus Traps During Async

Do not disable or remove interactive elements while loading in a way that strands keyboard focus. Ensure focus is predictably managed before, during, and after async operations.

MOBILE ACCESSIBILITY

Mobile Accessibility

Mobile introduces a distinct set of constraints: touch input, variable screen sizes, unpredictable orientations, and on-screen keyboards that reshape the viewport. Each of these affects how accessible an interface is in practice.

Touch Targets

Interactive elements must be at least 44×44 CSS pixels (WCAG 2.5.5). Insufficient touch targets cause errors for users with motor impairments and older users.

Zoom and Orientation

Do not set `user-scalable=no` in the viewport meta tag. Do not lock orientation. Users who need to zoom or rotate must be permitted to do so.

Single Column Layouts

Layouts must remain logical when reflow collapses content to a single column. Reading order, form flow, and navigation hierarchy all need verification at narrow widths.

MOBILE ACCESSIBILITY

Mobile Interaction Patterns

Patterns That Fail on Touch

- **Hover-only content** — tooltips, dropdowns, or popovers visible only on mouse hover are completely inaccessible on touch devices
- **Complex gestures** — swipe, pinch, and multi-finger actions must have single-tap or button-based alternatives
- **Drag-and-drop without alternative** — always provide a non-drag mechanism

Virtual Keyboard Behaviour

- Set the correct `inputmode` and `type` attributes to invoke the appropriate keyboard (numeric, email, URL)
- Ensure form elements are not obscured when the virtual keyboard appears — test on real devices
- Avoid positioning fixed elements that compete with the virtual keyboard viewport

In React and Angular, hover-state logic is often component-internal. Audit all tooltip and popover components for touch-accessible trigger alternatives.

CONTENT

Readable and Inclusive Content

Accessibility is not only a technical concern — content quality directly affects whether users can understand and act on what they are reading. Plain language, clear instructions, and helpful microcopy reduce cognitive load for everyone.

Use Plain Language

Aim for clear, direct sentences. Avoid jargon, passive constructions, and ambiguous pronouns. Plain language benefits users with cognitive disabilities, non-native speakers, and anyone reading quickly.

Make Instructions Specific

Replace vague guidance like "fill in the form correctly" with specific instructions: "Enter your date of birth in DD/MM/YYYY format." Concrete instructions prevent errors before they occur.

Write Helpful Microcopy

Button labels, confirmation messages, empty states, and error messages are all content accessibility concerns. Each should be written to reduce confusion, not create it — "Save changes" is better than "Submit".

CONTENT

Semantic SEO and Accessibility

Accessible structure and good SEO are complementary, not competing. The same semantic decisions that help screen reader users navigate a page also help search engines understand and index its content.

Headings Serve Both Users and Search

A logical heading hierarchy — one `<h1>`, followed by `<h2>` and `<h3>` sections — creates a document outline used by screen readers for navigation and by search engines for relevance scoring.

Meaningful Link Text

Links like "Read more" or "Click here" are meaningless out of context. Descriptive link text — "Read the WCAG 2.2 success criteria" — improves navigation for screen reader users browsing by links and improves crawlability for search engines.

Alt Text and Content Quality

Meaningful image alt text describes the content and function of an image. It improves accessibility for users with visual impairments and contributes to image search discoverability. Decorative images should carry `alt=""`.

FRAMEWORK PERSPECTIVE

Framework Perspective: React and Angular

Frameworks Do Not Guarantee Accessibility

React and Angular are rendering tools. They output HTML. The accessibility of that HTML depends entirely on what semantics the developer puts in. A component that looks correct in a browser can be completely broken in the accessibility tree.

- Check rendered HTML — not just JSX or templates
- Audit component library output with browser DevTools
- Verify roles, states, and properties in the accessibility tree

What to Check in Rendered Output

- **Semantic elements** — does a "button" component render a `<button>`?
- **Focus management** — is focus moved correctly after state changes?
- **ARIA attributes** — are `aria-expanded`, `aria-selected`, and `aria-live` applied and updated correctly?
- **Hidden content** — is `aria-hidden` used appropriately?

FINAL DELIVERY

Final Delivery Checklist

Before marking any interface as complete, verify these core dimensions. Each represents a different layer of the accessibility contract — structural, operational, and experiential.

✓ Structure is Semantic

- Landmarks, headings, and lists are correct
- Tables use proper markup and scope
- Images have appropriate alt text
- Forms have associated labels

✓ Keyboard Operation Works

- All interactive elements are reachable by Tab
- Focus order follows visual reading order
- Focus is never lost or hidden
- Custom widgets implement ARIA keyboard patterns

✓ Screen Reader Output is Understandable

- Names, roles, and values are accurate
- State changes are announced
- Dynamic content uses live regions
- No orphaned or misleading announcements

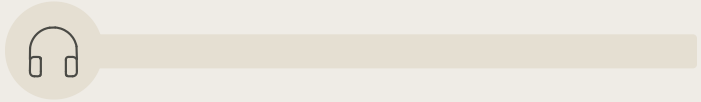
FINAL DELIVERY

Final Review Mindset

Tools and automated checks catch roughly 30–40% of accessibility issues. The rest require human judgement, manual keyboard testing, and direct screen reader use. Build these habits into your review process, not just your QA phase.

Test with Keyboard

Unplug the mouse. Navigate the entire interface using Tab, Shift+Tab, Enter, Space, and arrow keys. Every feature must be reachable and operable.



Test with NVDA or JAWS

Use NVDA (free) with Firefox or Chrome, or JAWS with Chrome. Browse by headings, landmarks, and links. Listen to what is announced on interaction.

Inspect the Rendered HTML

Open browser DevTools and inspect the actual DOM — not the component source. Check the Accessibility panel in Chrome or Firefox to view the computed accessibility tree for each element.

Key Takeaway

Accessible delivery is a full-interface responsibility.

Data, visuals, motion, mobile, and content are not separate concerns — they are interconnected layers of a single user experience. When any one layer fails, the whole experience can become unusable for someone.

The final interface must be **understandable** to users who cannot see it, **operable** by users who cannot use a mouse, and **robust** enough to work reliably with current and future assistive technologies.

The Three Tests

- Can a keyboard user complete every task?
- Does a screen reader user receive accurate, timely information?
- Does the rendered HTML support the intended semantics?

Accessibility implemented throughout delivery — not as a final audit — is what produces interfaces that genuinely work for everyone.